Solving issues introduced by relaxed template template parameter matching

Document #:	P3310R5
Date:	2024-11-18
Project:	Programming Language C++
Audience:	Evolution Working Group Core Working Group

Reply-to:	Matheus Izvekov <mizvekov@gmail.com>

Contents

§ 1 Introduction

This paper aims to address some lingering issues introduced with the adoption of P0522R0 into C++17, later made a defect report, which relaxed the rules on how templates are matched to template template parameters, but didn't concern itself with how partial ordering and overload resolution would be affected.

As a result, it invalidated several perfectly legitimate prior uses, creating compatibility issues with old code, forcing implementors to amend the rules, and overall slowing adoption.

We will point out two separate issues and their proposed solutions, with the intention that they be adopted as resolutions to [CWG2398], which is the core issue tracking this problem.

It is also suggested to bump the version for the feature testing macro.

§ 2 Changelog

§ 2.0.1 Since R4

— Clarifications and wording improvements.

§ 2.0.2 Since R3

— Included changes to overload resolution.

§ 2.0.3 Since R2

— Restricted effects of default argument deduction so it only applies to partial ordering.

§ 2.0.4 Since R1

— Improved solution's notations and explanation.

§ 2.0.5 Since R0

— Consequences of deducing default arguments for class templates are further explored and the rules tweaked to make it work consistently.

— Consequences of deducing default arguments for packs are further explored.

— The problem of inconsistent deductions is further explored.

— Added a paragraph on the consequences of mixing historical and P0522 affordances.

§ 3 Problem #1 - Default Arguments

Consider the following example:

```
template<class T1> struct A;

template<template<class T2> class TT1, class T3>
```

```
struct A<TT1<T3>>; // #1

template<template<class T4, class T5> class TT2, class T6, class T7>
struct A<TT2<T6, T7>> {}; // #2

template<class T8, class T9 = float> struct B;
template struct A<B<int>>;
```

Prior to [P0522R0], with the more strict rules, only the partial specialization candidate `#2` would be considered, since `B` has two template parameters, `T8` and `T9`, and the template template parameter in `#1` has only `T2` as a template parameter, so `#1` doesn't match.

After P0522R0, `#1` starts being accepted as a candidate, since `B` can be used just fine in place of `TT1` in the specialization `A<TT1<T3>>`: The default argument `float` for `T9` allows `B` to be specialized with just a single argument.

However, the rules for partial ordering operate considering only the candidates themselves, without access to `B`, which has the default template argument. When looking at only those candidates, it's not obvious the specialization of `TT1` would work, if that were to be replaced with a template with two parameters.

The only clue is the fact that these two candidates are being considered together: this must mean that default arguments are somehow involved.

But nonetheless, as things stand, these candidates cannot be ordered, resulting in ambiguity.

Maintaining the pre-P0522 semantics of picking `#2` would be ideal: Besides that it would be a compatible change, it is logical that this is the most specialized candidate.

§ 3.1 Solution to #1

When performing template argument deduction such that `P` and `A` are specializations of either class templates or template template parameters:

```
template <class T1, ... class Tn> TT1
template <class U1, ... class Un> TT2
P = TT1<X1, ..., Xn>
A = TT2<Y1, ..., Ym>
```

Under the current rules, `TT1` will be deduced as `TT2`.

This paper proposes that in this case, only during partial ordering, `TT1` will be deduced as a new template, synthesized from and almost identical to `TT2`, except that the specialization arguments (`Yn, ..., Ym`) will be used as default arguments in this new invented TT1, replacing any existing ones, effectively as if it had been written as:

```
template<class T1, ..., class Tn, class Tn+1 = Yn+1, ..., class Tm = Ym>
using TT1 = TT2<T1, ..., Tn, Tn+1, ..., Tm>
```

That is, all template arguments used in the specialization of TT2, which would not have been consumed had them been used in TT1, will be used to deduce the default arguments for TT2.

In particular, this means that had `Tn` been a parameter pack, no default arguments will be deduced for `TT2`, as that consumes all arguments.

When binding TT2 to TT1, the extra parameters are not accessible, since the specialization to TT1 is checked at template parse time, so the usage must match it's declaration.

So it's possible to conceptually simplify this, and think of this alias template as:

```
template<class T1, ..., class Tn> using TT1 = TT2<T1, ..., Tn, Yn+1, ...,
        Ym>
```

It's useful to think of these invented templates in terms of the equivalences proposed in [CWG1286], but the last proposed resolution for that core issue only recognizes the first form.

This paper is not proposing to standardize a syntax to create this implicit alias as written, outside of deduction, but for explanatory purposes, the following syntax will be used: `A:1<float>`.

Where this means, take the template name `A`, and create an alias from it by applying the following template arguments as defaults, starting from the second argument. Roughly equivalent to:

```
template<class T, class U> struct A {};
template <class T, class U = float> using invented_alias_A_1_float = A<T,
        U>;
```

This syntax will be used in the rest of this chapter to convey this adjustment.

§ 3.1.1 Class templates

Conversely, a template template parameter can also be deduced as a class template:

```
template <class T1, class T2 = float> struct A;

template <class T3> struct B;
template  <template  <class  T4>  class  TT1,  class  T5>  struct  B<TT1<T5>>;
        // #1
template <class T6, class T7>                       struct B<A<T6, T7>> {};
        // #2

template struct B<A<int>>;
```

When partially ordering `#1` versus `#2`, TT1 will be deduced as a synthesized template based on A, with `T7` as default argument for `T2`.

§ 3.1.2 Consistency

When a parameter is deduced against multiple arguments, there are issues related to consistency, both in deduced and non-deduced contexts.

Related to the former, given this example:

```
template<class T1, class T2> struct A;

template<template<class, class> class TT1, class T1, class T2, class T3>
struct A<TT1<T1, T2>, TT1<T1, T3>> {}; // #1

template<template<class> class UU1, class U1, class U2>
struct A<UU1<U1>, UU1<U2>>; // #2

template struct A<B<int>, B<int>>;
```

Prior to P0522, `#1` would be picked. This becomes ambiguous after `P0522`, and the rules proposed in this paper don't help.

The problem is that `UU1` will be deduced as both `TT1:1<T2>` and `TT1:1<T3>`, and these should in general be treated as different templates, so this would be taken as an inconsistent deduction under current rules.

It should be possible to make this work and keep the pre P0522 behavior, but the present paper doesn't go as far yet, and this is left for future work.

This would involve recognizing that we are deducing two templates against each other. `T2` and `T3` are both template parameters of it, and it's consistent that they could be deduced to refer to the same type.

Regarding consistency in non-deduced contexts, given this example:

```
template<class T> struct N { using type = T; };

template<class T1, class T2, class T3> struct A;

template<template<class, class> class TT1,
         class T1, class T2, class T3, class T4>
struct A<TT1<T1, T2>, TT1<T3, T4>,
         typename N<TT1<T1, T2>>::type> {};

template<template<class> class UU1,
         template<class> class UU2,
         class U1, class U2>
struct A<UU1<U1>, UU2<U2>, typename N<UU1<U1>>::type>;

template<class T8, class T9 = float> struct B;
template struct A<B<int>, B<int>, B<int>>;
```

This was accepted prior to P0522R0, but picking `T2` for the default argument of `TT1` will result in this example being accepted, but result in rejection of the symmetric example where `N<TT1<T1, T2>>` is replaced with `N<TT1<T1, T4>>`, which is also accepted pre-P0522.

This is a similar problem, and this paper does not propose a mechanism to either keep both working, neither to reject both as ambiguous. This is left for future work.

§ 3.1.3 Packs

As a particular consequence of this proposal, default arguments can be deduced for pack parameters, something which is not ordinarily possible to obtain.

This affects the partial ordering of the following example:

```
template<class T1, class T2 = float> struct A;
template<class T3> struct B;

template<template<class T4> class TT1, class T5>
struct B<TT1<T5>>; // #1

template<template<class  T6,  class  ...T7s>  class  TT2,  class  T8,  class
        ...T9s>
struct B<TT2<T8, T9s...>> {}; // #2

template struct B<A<int>>;
```

Before P0522, `#2` was picked. After P0522, this changed entirely, now `#1` is picked.

The proposed rules will restore the pre-P0522 behavior, picking `#2` again.

§ 4 Problem #2 - With vs Without Packs

Consider the following example:

```
template<template<class ...T1s> class TT1> struct A {};
template<class T2> struct B;
template struct A<B>; // #1

template<template<class T3> class TT2> struct C {};
template<class ...T4s> struct D;
template struct C<D>; // #2
```

Before [P0522R0], `#1` was valid, and `#2` was invalid.

After P0522R0, `#1` stayed valid, and `#2` became valid.

Now consider this partial ordering example:

```
template<class T1> struct A;

template<template<class ...T2s> class TT1, class T3>
struct A<TT1<T3>>; // #1

template<template<class T4> class TT2, class T5>
struct A<TT2<T5>> {}; // #2
```

```
template<class T6> struct B;
template struct A<B<int>>;
```

Before P0522R0, #2 was picked.

However, since the same rules which determine a template can bind to a template template parameter, are also used to determine one template template parameter is at least as specialized as another, and since P0522R0 made no special provisions in these rules for the latter, this example now becomes ambiguous.

But this is undesirable, because it is a breaking change, and also because logically #2 is more specialized.

The new paragraph inserted with P0522R0, which defines the 'at least as specialized as' rules in terms of function template rewrite, leads to this confusion:

There is a historical rule which allows packs in the argument to match non-packs in the parameter side.

The template argument list deduction rules accept packs in parameters and no packs in arguments, while the reverse is rejected, but in order to accept both #1 and #2 from the first example, both directions need to be accepted. An exception had to be carved out in 13.4.4 [temp.arg.template]/3 for this historical rule.

Note that this is a check for whether the parameter is at least as specialized as the argument, so the roles are switched around: P takes the role of the argument, and A takes the role of the parameter.

We propose a solution which will restore the semantics of the second example, by specifying that during partial ordering, packs must match non-packs only in the original direction which was accepted prior to P0522, which was a pack on the argument side matching non-packs in the parameter side.

Additionally, as a consequence of this exception, all implementations seem to reject combinations of the P0522 and historical rule affordances, which would otherwise be accepted individually, such as this example:

```
template<template<class, int, int...> class> struct A {};
template<class, char> struct B;
template struct A<B>;
```

Where there are packs in parameter side matching non-pack in argument side, and also matching of NTTPs of different type.

This is a very arbitrary restriction, and we propose to make it clear this is valid.

Now consider this example:

```
template <template <class...      > class TT1> struct A        { static
        constexpr int V = 0; };
template <template <class          > class TT2> struct A<TT2> { static
        constexpr int V = 1; };
template <template <class, class> class TT3> struct A<TT3> { static
        constexpr int V = 2; };

template <class ...          > struct B;
template <class              > struct C;
template <class, class       > struct D;
template <class, class, class> struct E;

static_assert(A<B>::V == 0);
static_assert(A<C>::V == 1);
static_assert(A<D>::V == 2);
static_assert(A<E>::V == 0);
```

Before P0522R0, this is accepted.

After P0522R0, this became wholly invalid: The partial specializations are not more specialized than the primary template. Also, the `A<B>` specialization becomes ambiguous.

With the proposed solution, the primary template becomes less specialized again.

But the other issue remains: The `A<B>` specialization stays ambiguous. Here the problem goes outside of the scope of partial ordering.

Now consider this example:

```
template <template <class   > class TT> void f(TT<int>); // #1
template <template <class...> class TT> void f(TT<int>); // #2

template <class   > struct A {};
template <class...> struct B {};
void test() {
  f(A<int>()); // selects #1
  f(B<int>()); // selects #2
}
```

After P0522, both calls became ambiguous, but that's so far only a partial ordering issue, which the solution so far breaks in favor of picking #1. But before P0522, overload resolution for the second call would only match candidate #2. Picking #2 is the desirable outcome here, since a pack seems to be a better match for a pack, and this would also avoid a breaking change.

§ 4.1 Solution to #2

We propose changing the deduction rules such that, only during partial ordering, packs matching to non-packs isn't accepted both ways, that it should only be accepted for a pack argument matching to parameter non-packs, which is the opposite direction it's normally accepted in the deduction of template argument lists.

We also propose removing the classic pack exception clause in 13.4.4 [temp.arg.template]/3, in favor of explicitly specifying that outside of partial ordering, packs matching to non-packs must be accepted both ways.

Lastly, we propose to introduce a new kind of notional conversion for the purpose of solving the overload resolution issue.

This is the same principle used in overload resolution, where overloads are ranked according to the conversions involved when matching arguments to their parameters. Some conversions are considered better than others, which may lead to certain candidates being considered better than others.

This new conversion would be defined by the presence of a pack on the argument side matching a non-pack on the parameter side, whenever packs matching non-packs would also be allowed in the opposite direction. A candidate would be better when this conversion is not used.

§ 5 Wording

Add a new paragraph after 13.7.7.3 [temp.func.order]/5:

5 [Note 3: Since, in a call context, such type deduction considers only parameters for which there are explicit call arguments, some parameters are ignored (namely, function parameter packs, parameters with default arguments, and ellipsis parameters).

6 When matching template template parameters as described in [temp.deduct.type], if P is a specialization of a template template parameter, the specialized parameter will be deduced as a modified template, except that elements of the template argument list in A are used as the default arguments for the modified template's template parameters, starting from the first parameter in A which has a correspondence in P, up to the last parameter in A which has a corresponding argument.

[Example:

```
template <class, class = float> struct A {};

template <template <class> class TT, class T> void f(TT<T>);
        // #1
template <template <class, class> class TT, class T, class U> void f(TT<T,
        U>); // #2

template <template <class> class TT, class T> void g(TT<T>); // #3
template <class T, class U> void g(A<T, U>);                 // #4

void test(A<int> a) {
  f(a); // selects #2
  g(a); // selects #4
}
```

— end example]

Remove 13.4.4 [temp.arg.template]/3, moving the non-crossed-out section to the next paragraph, with further modifications:

3 A template-argument matches a template template-parameter P when P is at least as specialized as the template-argument A. In this comparison, if P is unconstrained, the constraints on A are not considered. ~~If P contains a template parameter pack, then A also matches P if each of A's template parameters matches the corresponding template parameter in the template-head of P. Two template parameters match if they are of the same kind (type, non-type, template), for non-type template-parameters, their types are equivalent ([temp.over.link]), and for template template-parameters, each of their corresponding template-parameters matches, recursively. When P's template-head contains a template parameter pack ([temp.variadic]), the template parameter pack will match zero or more template parameters or template parameter packs in the template-head of A with the same type and form as the template parameter pack in P (ignoring whether those template parameters are template parameter packs).~~

4 A template-argument A matches a template template-parameter P when P is at least as specialized as A~~. In this comparison, if P is unconstrained, the constraints on A are not considered~~, ignoring constraints on A if P is unconstrained. A template template-parameter P is at least as specialized as a template template-argument A if, given the following rewrite to two function templates, the function template corresponding to P is at least as specialized as the function template corresponding to A according to the partial ordering rules for function templates. Given a~~n~~ TTP invented class template ([temp.deduct.type]) X with the template-head of A (including default arguments and requires-clause, if any):

…

If the rewrite produces an invalid type, then P is not at least as specialized as A.

[Note: [temp.deduct.type] has a special case for this deduction - end note]

[Example:

```
template<class T1> struct A;
template<template<class ...> class TT, class T> struct A<TT<T>>; // #1
template<template<class    > class TT, class T> struct A<TT<T>>; // #2

template<class> struct B;
template struct A<B<int>>; // selects #2

template <template <class...> class TT> struct C; // #3
template <template <class> class TT> struct C<TT>; // OK, more specialized
        than #3

template <template <class> class TT> struct D; // #4
template <template <class...> class TT> struct D<TT>; // error: not more
        specialized than #4
```

— end example]

[Example:

```
template<template<class, char, char...> class> struct A {};
template<class, int> struct B;

template struct A<B>; // OK, `char` matches 'int', pack matching non-pack
        is accepted.
```

— end example]

Add a new paragraph before 13.10.3.6 [temp.deduct.type]/9, subsequently modifying it and adding a bulleted list:

9 A *TTP invented class template* is an invented class template used for type deduction when determining whether a template argument matches a template template-parameter ([temp.arg.template]).

10 If P has a form that contains <T>~~or~~, <i>, or <TT>, then each argument P_i of the respective template argument list of P is compared with the corresponding argument A_i of the corresponding template argument list of A. If the template argument list of P contains a pack expansion that is not the last template argument, the entire template argument list is a non-deduced context. ~~If P_i is a pack expansion, then the pattern of P_i is compared with each remaining argument in the template argument list of A. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by P_i. During partial ordering, if A_i was originally a pack expansion:~~

(10.1) If P_i is a pack expansion, then the pattern of P_i is compared with each remaining argument in the template argument list of A. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by P_i. ~~During partial ordering, if A_i was originally a pack expansion~~ A *strict pack match* occurs when deducing the argument list of the specializations of a TTP invented class template ([temp.deduct.type]), and A_i is not a pack expansion. During partial ordering, if A_i was originally a pack expansion:

(10.1.1) - if this is a strict pack match, template argument deduction fails.

(10.1.2) - otherwise, if P does not contain a template argument corresponding to A_i then A_i is ignored;

(10.1.3) - otherwise, if P_i is not a pack expansion, template argument deduction fails.

(10.2) When deducing the argument list of the specializations of a TTP invented class template ([temp.deduct.type]), if A_i is a pack expansion, and there is at least one remaining argument in P, then the pattern of A_i is compared with each remaining argument in the template argument list of P. Each comparison deduces template arguments for subsequent positions in the template parameter packs expanded by A_i. If P_i was originally a pack expansion:

(10.2.1) - if A does not contain a template argument corresponding to P_i then P_i is ignored;

(10.2.2) - otherwise, template argument deduction fails.

[Note: The last two subclauses are the reverse of each other. - end note]

Add a new paragraph after 12.2.4.1 [over.match.best.general]/2.3:

(2.4) Both F1 and F2 are function template specializations, and F1 was not a strict pack match ([temp.deduct.type]) as deduced from the arguments of the call, and F2 was.

[Example:

```
template <template <class   > class TT> void f(TT<int>); // #1
template <template <class...> class TT> void f(TT<int>); // #2

template <class   > struct A {};
template <class...> struct B {};
void test() {
  f(A<int>()); // selects #1
  f(B<int>()); // selects #2
}
```

— end example]

Add a new example to 13.7.6.3 [temp.spec.partial.order]/1:

1 For two partial specializations, the first is more specialized than the second if, given the following rewrite to two function templates, the first function template is more specialized than the second according to the ordering rules for function templates:

[Example 3:

```
template<template<class...> class> struct A;     // #1
template<template<class> class TT> struct A<TT>; // #2

template<class...> struct B;
template<class> struct C;

template struct A<B>; // selects #1
template struct A<C>; // selects #2
```

— end example]

When instantiating A<B>, matching B to #1 is not a strict pack match ([temp.deduct.type]), while #1 is, so #1 is a better match.

§ 6 Acknowledgements

Many thanks to those who have reviewed, contributed suggestions, and otherwise helped the paper reach it's current state: Arthur O'Dwyer, Brian Bi, Corentin Jabot, James

Touton, Jens Maurer, Richard Smith.

§ 7 References

[CWG1286] Gabriel Dos Reis. 2011-04-03. Equivalence of alias templates.
https://wg21.link/cwg1286

[CWG2398] Jason Merrill. 2016-12-03. Template template parameter matching and deduction.
https://wg21.link/cwg2398

[P0522R0] James Touton, Hubert Tong. 2016-11-11. DR: Matching of template template-arguments excludes compatible templates.
https://wg21.link/p0522r0